System design guide

The idea here is simple. We sit together and work through each design question the way you would actually face it, not just jump straight to the answer. For every question I first explain what the person asking is really trying to see in you, then how I would slowly build the design, clarifying the requirements first, doing a rough estimate, and only then deciding the pieces, giving the reason and the trade-off behind each choice. Wherever a picture helps there is a high-level design, and where the internals matter a low-level one also. At the end, the follow-up questions that usually come next, with short answers. Throughout, the depth is pitched at a senior 2026 level.

Contents

1. Design a URL shortener (e.g., bit.ly). Discuss data model, unique ID generation, and handling redirects.

What this question is really testing. This one looks like a warm-up, but quietly it tells the person a lot about you. What they want to see is whether you scope the problem properly before writing anything, whether you notice that this is a read-heavy, low-latency system and shape the read path for it, and most of all, how you reason about the real crux here, which is unique ID generation. That is the part where the difference between a junior and a senior shows up. The usual follow-ups they keep ready are custom aliases, analytics, expiry, collisions, scaling the datastore, and 301 v/s 302.

How I would drive it. Since jumping straight into a design usually goes wrong, I first settle what we are actually building. On the functional side it is just two things: take a long URL and give back a short code, and when someone hits the short code, redirect them to the original. Optionally custom aliases, expiry, and click analytics. The non-functional side is what really shapes the whole design, so I spend a moment there: this system is very heavily read-skewed, the redirects far outnumber the creates, roughly 100 to 1, the redirect must be fast and always available, and the code should stay short.

After that a quick back-of-the-envelope, only to justify the choices, not to show off arithmetic. Say we get around 100 million new URLs a month. That is only about 40 writes a second, but at 100 to 1 it becomes roughly 4,000 redirects a second, and over five years about 6 billion stored URLs, a few TB. Two things fall out of this straightaway: the write path is trivial, and the whole game is the read path plus a short code space. With base62 (a to z, A to Z, 0 to 9), a 7-character code gives 62 to the power 7, about 3.5 trillion combinations, so 7 characters is more than enough.

Data model. Since the only access pattern is a point lookup by the short code, this is basically a key-value lookup, so I would not force a heavy relational schema on it. The record looks like this:

Field	Type	Notes
`short_code`	string (primary key)	7-char base62, the lookup key
`long_url`	string	the destination
`created_at`	timestamp	
`expires_at`	timestamp	optional TTL
`owner_id`	string	optional, for user accounts

Because the lookup is a pure point lookup by `short_code`, a NoSQL key-value store like DynamoDB or Cassandra fits nicely and scales sideways for the read volume. A relational table with the primary key on `short_code` is also perfectly fine at this scale, so I would let the read volume and the team's comfort decide.

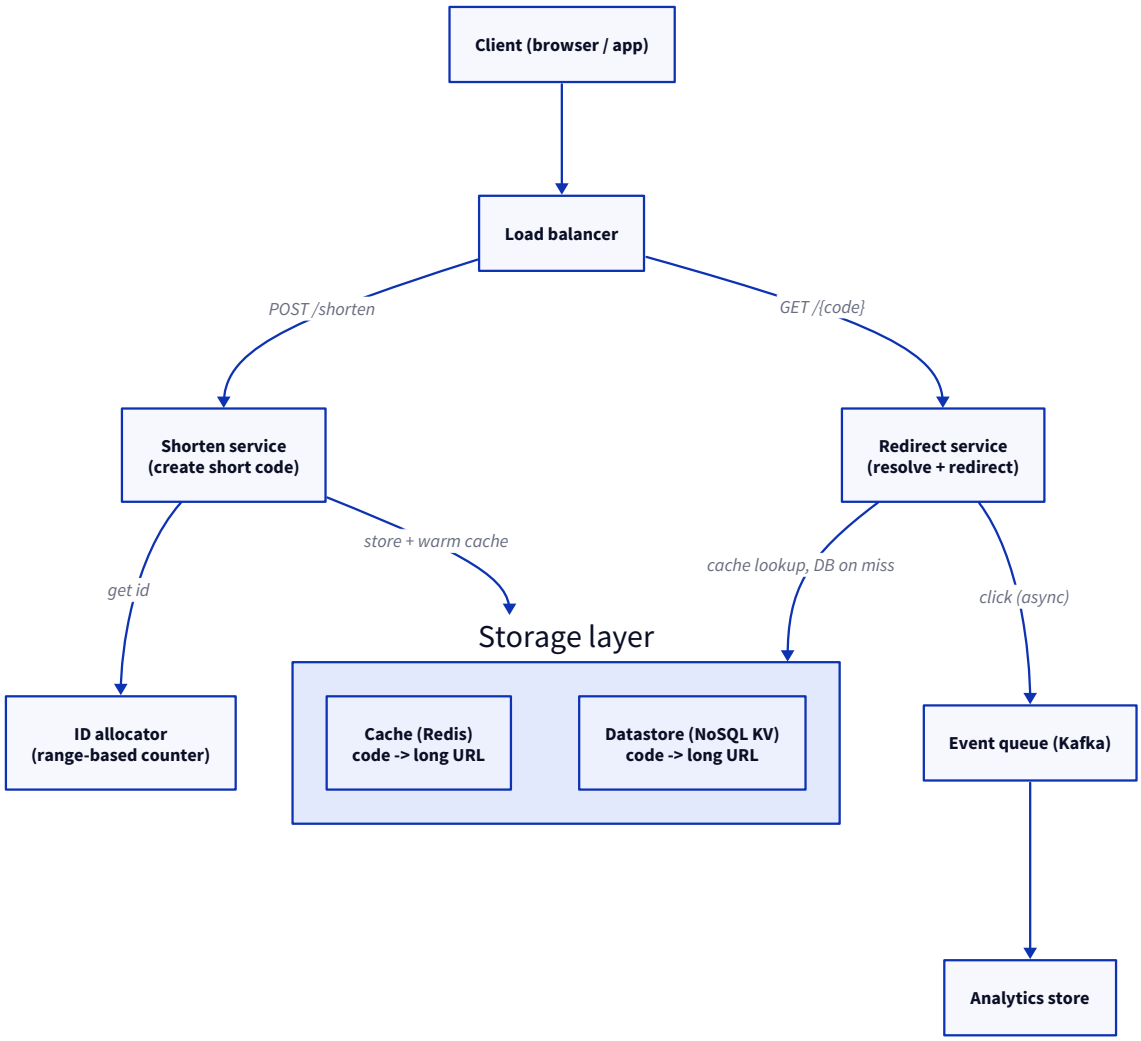
Unique ID generation (the crux). There are a few ways to do this, and each one has a real trade-off, so let me lay them side by side:

Approach	Pro	Con
Hash(URL) + base62	Deterministic, dedups the same URL	Collisions to detect and resolve
Auto-increment + base62	Unique by construction, short	Single counter becomes a bottleneck
Range-based distributed counter	Unique, scales, short codes	Slightly more moving parts
Random code + collision check	Simple	A DB check per create, retries on clash
UUID	No coordination needed	Too long, not user-friendly

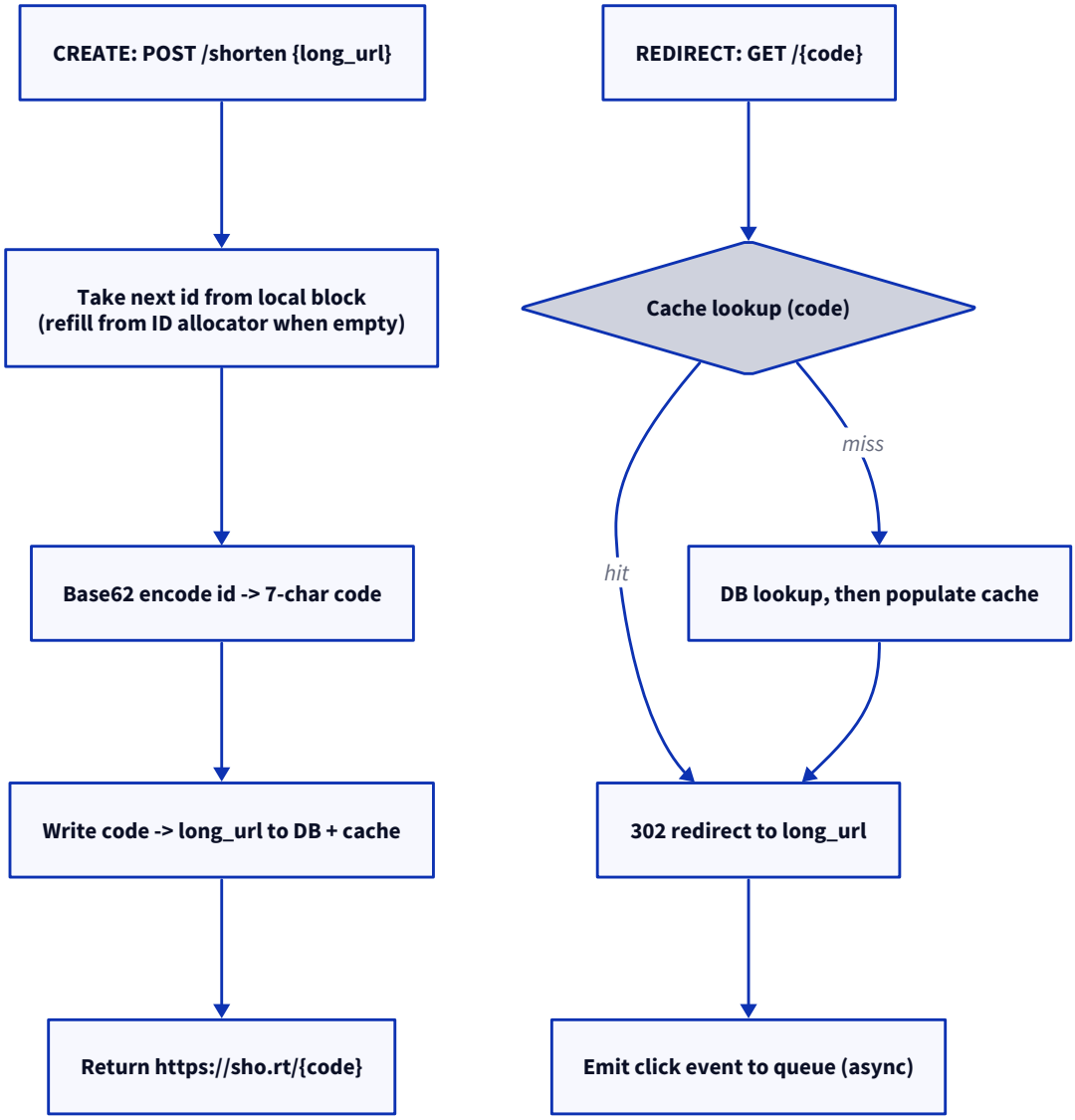
My pick is the range-based distributed counter. The way it works is that a central allocator hands each app server a block of IDs, say 1,000 at a time, and the server gives them out locally and only goes back to the allocator when its block is finished. Each ID is then base62-encoded into the short code. This gives us guaranteed uniqueness with no coordination on every request and no collisions at all, and the codes stay short. The trade-off v/s hashing is worth saying out loud: hashing dedups nicely, the same long URL always gives the same code, but then you must handle collisions; the counter never collides but it also does not dedup, so if dedup matters I would keep a separate `long_url -> code` lookup on the side. I would name this trade-off rather than pretend there is one correct answer.

Handling redirects (the read path). A redirect is just `GET /{code}`, a lookup, and then an HTTP redirect. Two decisions matter here. First, 301 v/s 302. A 301 (permanent) gets cached by the browser and the CDN, so it takes load off your servers, but then you lose the per-click analytics and you can never change the target. A 302 (temporary) comes back to your server on every click, which is exactly what you want if analytics and a changeable target matter. Most shorteners choose 302 for this reason, and so would I, unless analytics is genuinely not needed. Second, and this is the real scaling move: keep a cache (Redis) in front of the datastore, keyed by `code -> long_url`. Since the hot links dominate, the hit ratio stays high and the redirect becomes just a memory lookup. A CDN can sit in front of that also.

High-level design.



Low-level design (the two flows). On create, the server takes the next ID from its local block, refilling from the allocator when it is empty, base62-encodes it, writes `code -> long_url` into the datastore, warms the cache, and returns the short URL. On redirect, it checks the cache first; on a hit it sends the 302 straightaway, and on a miss it reads the datastore, populates the cache, and then redirects, sending the click event to a queue asynchronously so that analytics never slows down the redirect.



Follow-ups they will push on.

- **Custom aliases?** Let the user pick a code, check it is free, and store it alongside the generated ones.
- **How do you handle collisions?** The counter approach avoids them; if you hash instead, detect the clash and rehash.
- **Analytics without slowing redirects?** Send the click event to Kafka and process it asynchronously, never block the 302.
- **Expiry?** An `expires_at` field plus a cache TTL, with lazy or background cleanup of expired rows.
- **Scaling the datastore?** Shard by a hash of the short code; a NoSQL KV store does it for you.
- **Is the counter a single point of failure?** Run the allocator HA; the local ID blocks keep creates going through a short outage.
- **Abuse (malicious URLs)?** Validate and scan the destinations, and rate-limit creation per user or IP.

2. Design a rate limiter for an API. Explain token bucket vs. leaky bucket and approaches for distributed enforcement.

What this question is really testing. Two things, really. First, whether you know the standard algorithms and their trade-offs, which the question is asking for directly. Second, and this is where the seniors pull ahead, whether you can handle distributed enforcement, because the limit has to hold across many app servers, and that is a shared-state, atomicity, and consistency problem. The follow-ups they keep ready are the fixed-window boundary burst, what to key on, what to return, fail-open v/s fail-closed, and Redis becoming a bottleneck or single point of failure.

How I would drive it. Let me first clarify the shape. What are we limiting by? Usually per API key or per user, sometimes per IP. What is the limit? Say 100 requests per minute per key. And what happens when someone crosses it? We reject with a 429 Too Many Requests and a `Retry-After` header, or we throttle. The constraint that shapes everything is this: since the rate limiter sits on the hot path of every single request, it must add very little latency, and it must enforce one shared limit across all the app instances, which is the distributed part. One more thing I would settle early, strict v/s approximate, because perfect accuracy is costly and most APIs are quite happy with a small margin.

The algorithms. The ones worth knowing:

Algorithm	How it works	Trade-off
Fixed window	Count per fixed window (per minute), reset each window	Simplest, but a boundary burst allows up to 2x at the window edge
Sliding window log	Store each request's timestamp, count those in the last window	Accurate, but memory-heavy (a timestamp per request)
Sliding window counter	Weighted blend of current + previous window counts	Cheap and smooths the boundary problem, a good default
Token bucket	Tokens refill at a fixed rate up to a capacity; each request takes one	Allows bursts up to bucket size, limits the average rate
Leaky bucket	Requests queue and "leak" out at a fixed rate; overflow is dropped	Smooths output to a constant rate, no bursts downstream

Token bucket v/s leaky bucket (the direct ask). These two optimise for opposite things, so let me put them together:

	Token bucket	Leaky bucket
Model	Tokens accumulate; a request spends one	Requests enter a queue that drains at a fixed rate
Bursts	Allowed, up to the bucket capacity	Smoothed away, output is a steady stream
Optimises for	Limiting the average rate while tolerating spikes	Enforcing a constant rate to protect a downstream
Best when	Public APIs where the odd burst is fine (better for the user)	A fragile downstream that needs a steady flow

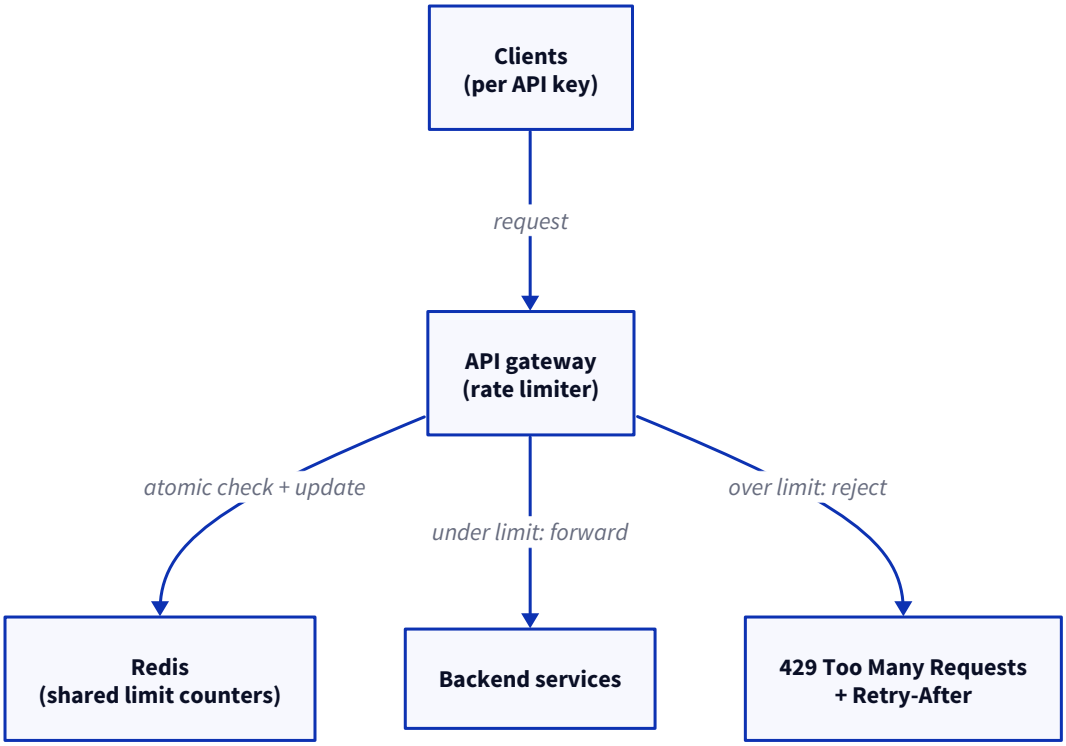
So the one line to remember is that the token bucket permits bursts and the leaky bucket flattens them. For a normal API I would go with the token bucket, since users do legitimately burst and it is forgiving; I would reach for the leaky bucket only when some downstream genuinely needs a constant input rate.

Distributed enforcement (the hard part). Here is the trap. If each of the N app servers keeps its own local counter, the effective limit quietly becomes N times what you intended. So we need a shared view of the count. The options are:

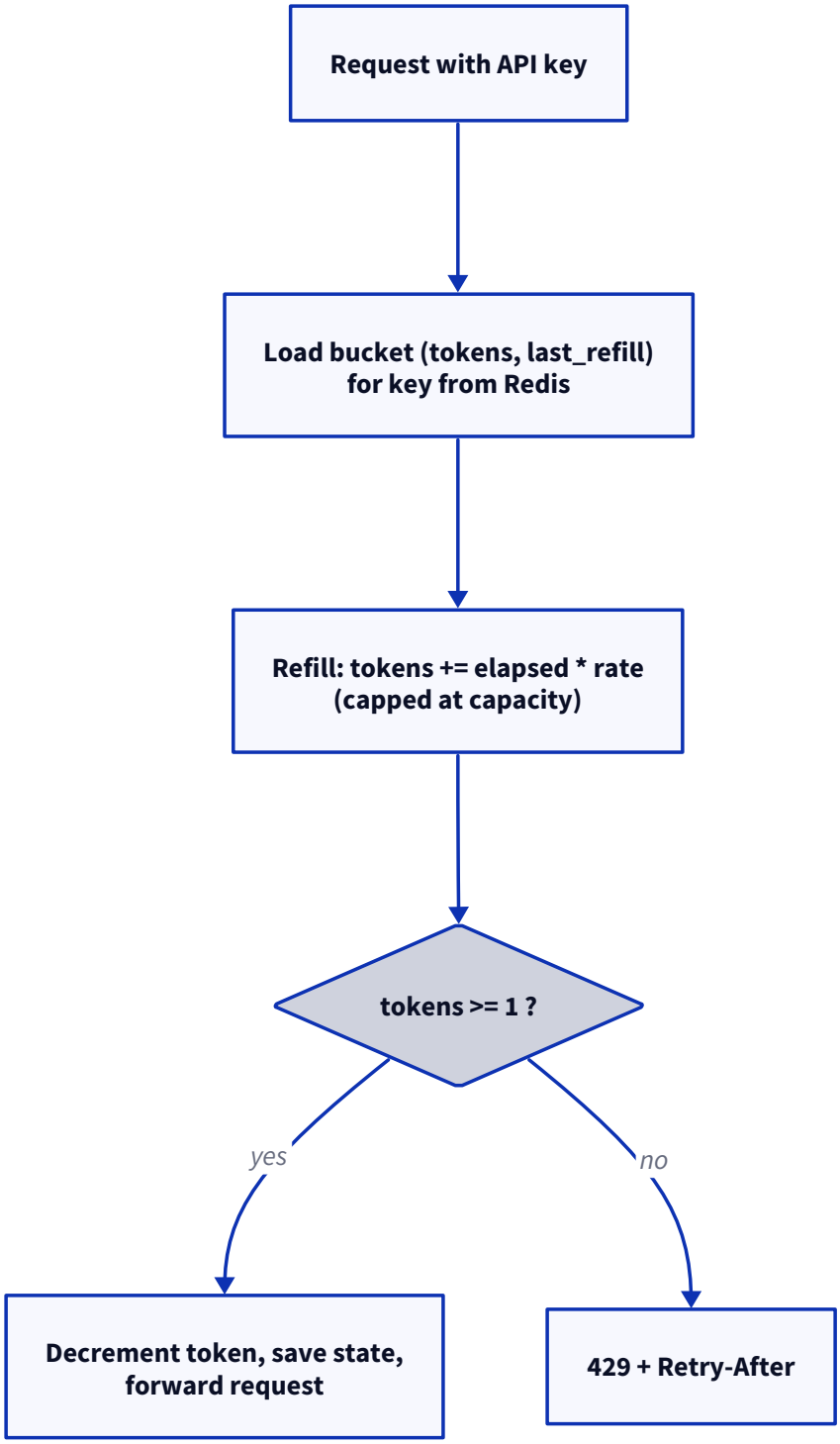
- **Centralised store (Redis).** Every server checks and updates a shared counter in Redis. The critical detail is atomicity, because a naive read-then-write races between servers, so you make the whole check-and-decrement atomic with `INCR` / `EXPIRE`, or for the token bucket a small Lua script that runs on Redis in one shot. This is the standard approach. The trade-off is a network hop per request and Redis becoming a bottleneck and a single point of failure, so you run it as a cluster and decide fail-open v/s fail-closed for when it goes down.
- **Local counters with a share of the limit.** Each server enforces `limit / N` locally, with no per-request network hop, so it is fast, but it is approximate and wastes budget when the traffic is uneven.
- **Enforce at the API gateway.** Run the limiter once at the edge, in the gateway or a dedicated rate-limit service, rather than inside every app server, so there is one natural place holding the shared state.

My recommendation is to enforce at the gateway, using a Redis-backed token bucket updated by an atomic Lua script, keyed by the API key. On a Redis outage I would fail open, since letting traffic through is better than blocking everyone, unless the limit is a hard security or cost boundary, in which case I would fail closed. Either way I would say that trade-off out loud.

High-level design.



Low-level design (the distributed token-bucket check). For each request the gateway runs one atomic operation against Redis: it loads the bucket state for the key, the current tokens and the last refill time, refills based on the elapsed time up to the capacity, and if there is at least one token it decrements and allows, otherwise it rejects with a 429 and a `Retry-After`. Doing the whole thing in one Lua script is what removes the read-modify-write race between the servers.



Follow-ups they will push on.

- **Why not fixed window?** The boundary burst: 100 at 0:59 plus 100 at 1:00 is 200 in two seconds. Use sliding window or token bucket.
- **What do you key on?** API key or user id for fairness, IP as a fallback for anonymous traffic.
- **What do you return?** 429 with `Retry-After` and remaining/reset headers so clients back off politely.

- **Redis is down, now what?** Fail open for a soft limit, fail closed for a hard security or cost limit.
- **Redis latency on the hot path?** Keep it in-region, use one atomic script, and add a small local pre-check.
- **Multiple tiers of limits?** Layer them, a per-second burst plus a per-day quota, each its own bucket.

3. Design an authentication system for a single-page app. Cover token types (JWT vs. opaque), refresh tokens, and session revocation.

What this question is really testing. Whether you understand the stateless v/s stateful trade-off between JWT and opaque tokens, whether you know the access-plus-refresh-token pattern that most real single-page apps use, and the two hard, app-specific problems: where to keep the tokens safely inside a browser, since XSS and CSRF are the real threats there, and how to revoke a session when a JWT stays valid until it expires. The follow-ups they keep ready are XSS v/s CSRF, short expiry, refresh-token rotation and theft, and instant logout.

How I would drive it. Let me clarify the setup first. It is a browser app talking to an API, probably several services, users log in with credentials or through an identity provider, and we need to authenticate every request, keep users logged in without asking for the password again and again, and be able to log them out or kill a compromised session. The environment matters most here, because it is a browser, so I am really designing against XSS, a script stealing the token, and CSRF, a forged request riding on a cookie, and I want the request path to stay fast across the several services.

Token types (JWT v/s opaque), the crux. These sit at the two ends of the stateless-stateful trade-off:

	JWT (self-contained)	**Opaque token**
What it is	Signed token carrying claims (user, roles, expiry)	A random string; the session lives on the server
Validation	Verify the signature locally, no lookup	Look it up in a store on every request
State	Stateless, scales across services	Stateful (needs a shared store)
Revocation	Hard, valid until it expires	Easy, delete the server-side record
Cost	Larger token; claims can go stale	A store lookup per request

So the tension is clear: the JWT is stateless and scales well but is hard to revoke, while the opaque token is easy to revoke but needs a lookup. Since we want both, the usual way is to combine them.

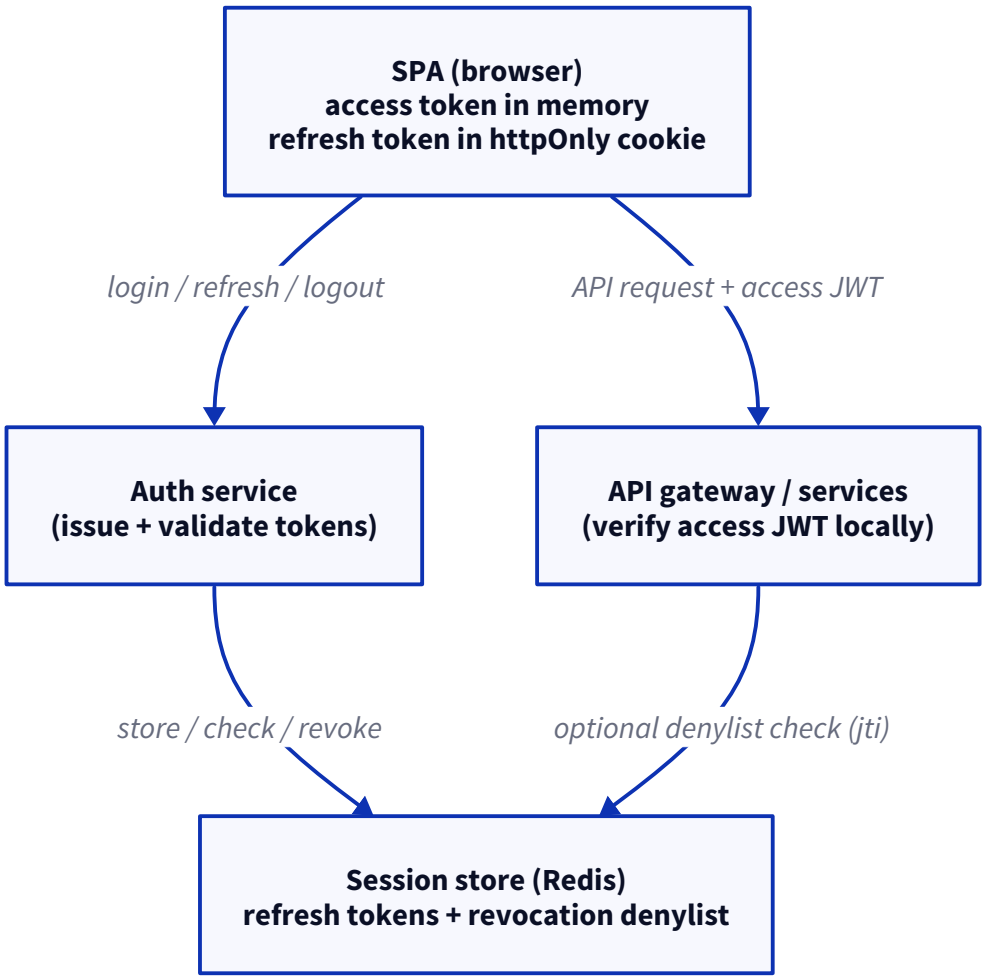
The access plus refresh pattern (how I would build it). We issue two tokens. A short-lived access token, a JWT of about 15 minutes, goes on every API request and is verified locally by each service, so the hot path stays stateless and fast, and the short life keeps the damage small if it leaks. A long-lived refresh token, opaque and stored on the server for a few days, is used only to get a new access token when the old one expires. Since the refresh token is checked against the store, we can revoke it. So the access token buys us statelessness on the hot path, and the refresh token buys us revocation.

Session revocation (the direct ask). A JWT access token cannot be pulled back mid-life, so we keep it short, around 15 minutes, to keep the window small. To force a logout or kill a compromised session, we revoke the refresh token by deleting it from the store, after which the user cannot renew and the current access token anyway dies within minutes. If we need to kill an access token instantly, say during a security incident, we keep a small denylist of revoked token ids (jti) in Redis and check it per request, accepting that this brings back a

lookup. And I would use refresh-token rotation: each refresh issues a fresh refresh token and invalidates the old one, so if an already-used token turns up again, that is a sign of theft and we revoke the whole family.

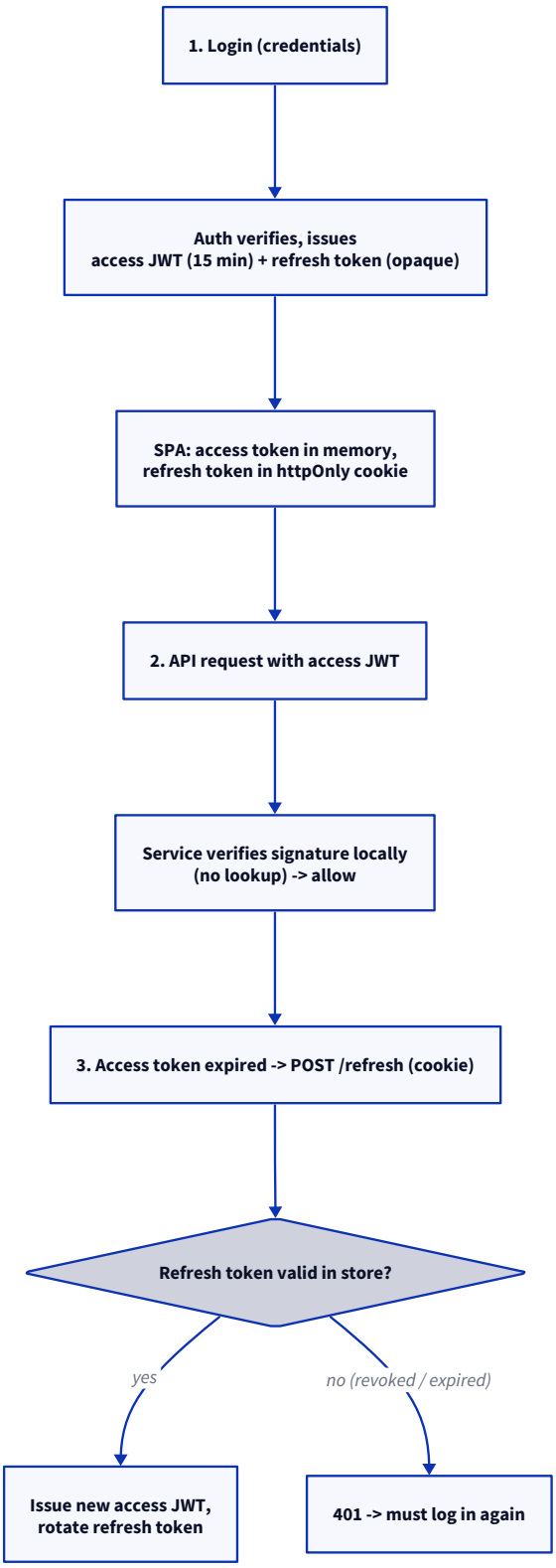
Token storage in the browser (the app-specific decision). This is where single-page apps usually get hacked, so it needs care. Since `localStorage` can be read by any injected script, it is exposed to XSS, so I would not put tokens there. The safer pattern is to keep the refresh token in an httpOnly, Secure, SameSite cookie, which JavaScript cannot read, so XSS cannot steal it, and SameSite plus a CSRF token handles the CSRF side, while the access token stays in memory, a plain JS variable, so it is never persisted where a script can pick it up. The one trade-off is that a page reload loses the in-memory access token, so on load the app quietly calls `/refresh` with the cookie and gets a fresh one.

High-level design.



Low-level design (the token lifecycle). On login the auth service checks the credentials and issues a short access JWT plus an opaque refresh token, stored in Redis and set as an httpOnly cookie, and the app keeps the access token in memory. On each API request the service verifies the JWT signature locally and allows it, no lookup. When the access token expires, the app calls `/refresh`, the auth service checks the refresh token in

the store, and if it is valid it issues a new access token and rotates the refresh token, otherwise it returns a 401. Revocation is simply deleting the refresh token from the store.



Follow-ups they will push on.

- **XSS v/s CSRF?** XSS steals tokens (mitigate with an httpOnly cookie, no `localStorage`); CSRF forges requests (mitigate with a SameSite cookie plus a CSRF token).

- **Why keep the access token so short?** A JWT cannot be revoked mid-life, so a short expiry keeps the damage from a leak small.
- **Refresh token stolen?** Rotation detects reuse of an old token and revokes the whole family.
- **How do you log out everywhere?** Delete all of the user's refresh tokens from the store.
- **Why not opaque tokens everywhere?** A lookup on every request across many services does not scale; the JWT keeps the hot path stateless.
- **Do you build this yourself?** Usually not, you use an identity provider (OAuth 2.0 / OIDC), and this becomes the token model behind it.

4. Design file upload handling for a web app. Cover direct-to-cloud uploads, resumable uploads, virus scanning, and metadata storage.

What this question is really testing. Whether you know the one move that makes uploads scale, which is to not send the file bytes through your own app servers, and then the pieces around it: resumable uploads for large files on a shaky network, asynchronous virus scanning that does not make the user wait, and how you model the metadata and the file lifecycle. The follow-ups they keep ready are presigned-URL security, multipart for large files, what happens when a scan fails, dedup, and serving the downloads.

How I would drive it. Let me clarify the shape first. What kind of files and what size range, since images v/s large videos changes whether resumable really matters, who can upload and how access is controlled, whether we scan for viruses, which here we do, and whether downloads are also in scope, where I would focus on upload and just note a CDN for downloads. The load picture is many uploads at the same time, some of them large, and often on unreliable client networks.

Direct-to-cloud uploads (the key move). The natural instinct is to route the file through your backend, and that is exactly the trap, because it burns your servers' bandwidth and memory and simply does not scale. So instead the client asks the backend for a presigned URL and uploads directly to the object storage, S3 or GCS, and the backend only handles the small presign request and the metadata. This pushes the heavy bandwidth onto the cloud provider, which is built for it. The presigned URL is time-limited and scoped to one key and one operation, so it is safe to hand to the client.

Resumable uploads. For a large file on a shaky network, a single PUT that fails means starting again from zero, which is quite painful on mobile. A resumable upload splits the file into chunks, uploads them, keeps track of which ones succeeded, and on a failure retries only the failed chunk, before a final "complete" call stitches them together. This is what S3 multipart, GCS resumable uploads, and the tus protocol give you. The trade-off is more moving parts, tracking parts and completing or aborting, but above a few tens of MB it is genuinely needed.

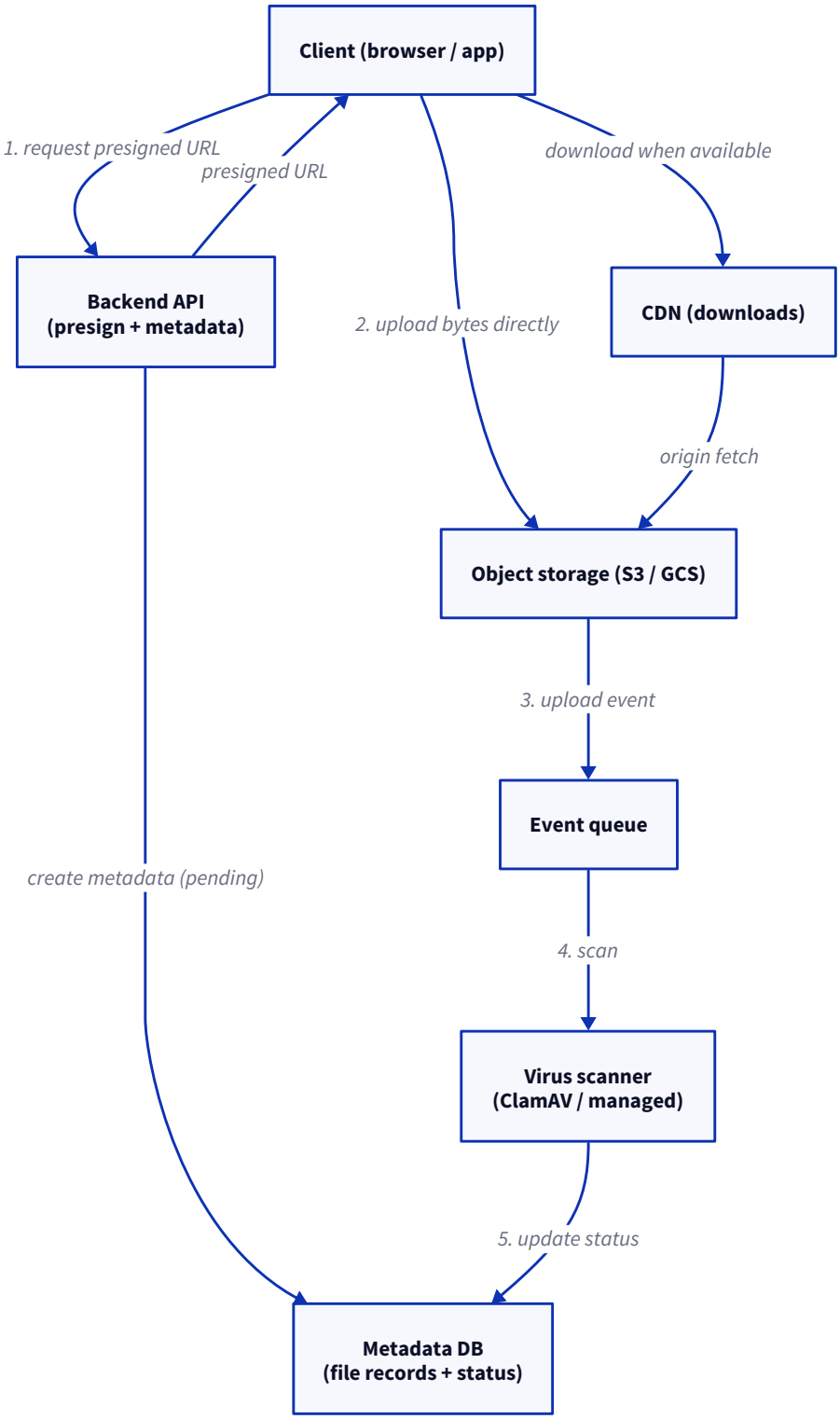
Virus scanning (asynchronous). We never make the user wait on a scan. The file lands in the storage in a pending or quarantined state, a storage event goes to a queue and triggers a scanner, ClamAV or a managed service, and only on a clean result is the file marked available; an infected file is deleted or quarantined and the owner is told. The file is never served until it has passed the scan, and it is the metadata status that enforces this.

Metadata storage. The bytes live in the object storage, and the database keeps one record per file, which is the source of truth for access control, listing, and the scan lifecycle:

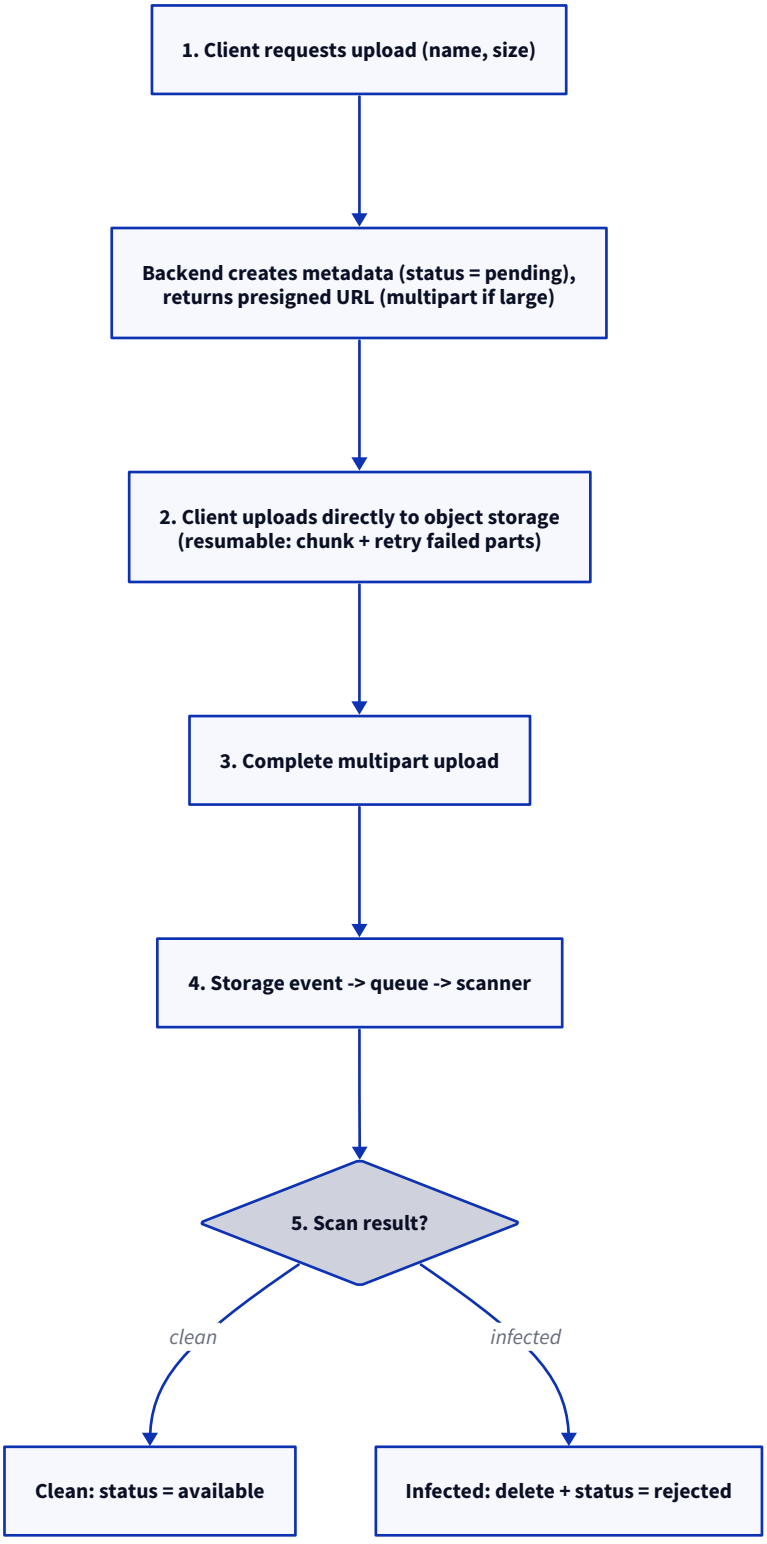
Field	Notes
`file_id`	primary key
`owner_id`	for access control
`filename`	original name
`content_type`	MIME type
`size`	bytes
`storage_key`	pointer into object storage
`status`	pending / scanning / available / rejected
`checksum`	integrity and dedup
`created_at`	timestamp

The `status` field is what drives the whole lifecycle, and it is the reason a half-uploaded or unscanned file can never be served.

High-level design.



Low-level design (the upload and scan flow). The client asks the backend to start an upload, the backend writes a metadata row as `pending` and returns a presigned URL, a multipart one for large files. The client uploads directly to the object storage, chunking and retrying failed parts, and then calls complete. A storage event flows through a queue to the scanner; a clean result flips the status to `available`, and an infected one deletes the object and marks it `rejected`. The client polls or gets notified, and the file becomes downloadable, through a CDN, only once it is available.



Follow-ups they will push on.

- **Why not upload through your servers?** Bandwidth and memory do not scale; presigned URLs push the bytes straight to storage.
- **How do presigned URLs stay secure?** Short expiry, scoped to one key and operation, and issued only after the user is authorised.

- **Large files?** Multipart or resumable upload, retrying only the failed part rather than the whole file.
- **Scan fails or times out?** Keep the file quarantined, never `available`, retry, and alert if it keeps failing.
- **Deduplication?** Hash the file; if the checksum already exists, point the new metadata row at the same stored object.
- **Serving downloads?** A CDN in front of the storage, with presigned URLs or signed cookies for private files.

5. Design a notification system for email. Discuss delivery guarantees, retry strategies, and prioritization.

What this question is really testing. Whether you understand that with email you own very little of the delivery pipeline, a provider and the recipient's mail server own most of it, so you design around at-least-once to the provider plus best-effort to the inbox, asynchronous bounce and complaint feedback, retries that tell soft from hard failures, and a priority separation so that a marketing blast never delays a password reset. All three notification questions share one pipeline, so the interesting part here is the email-specific reality.

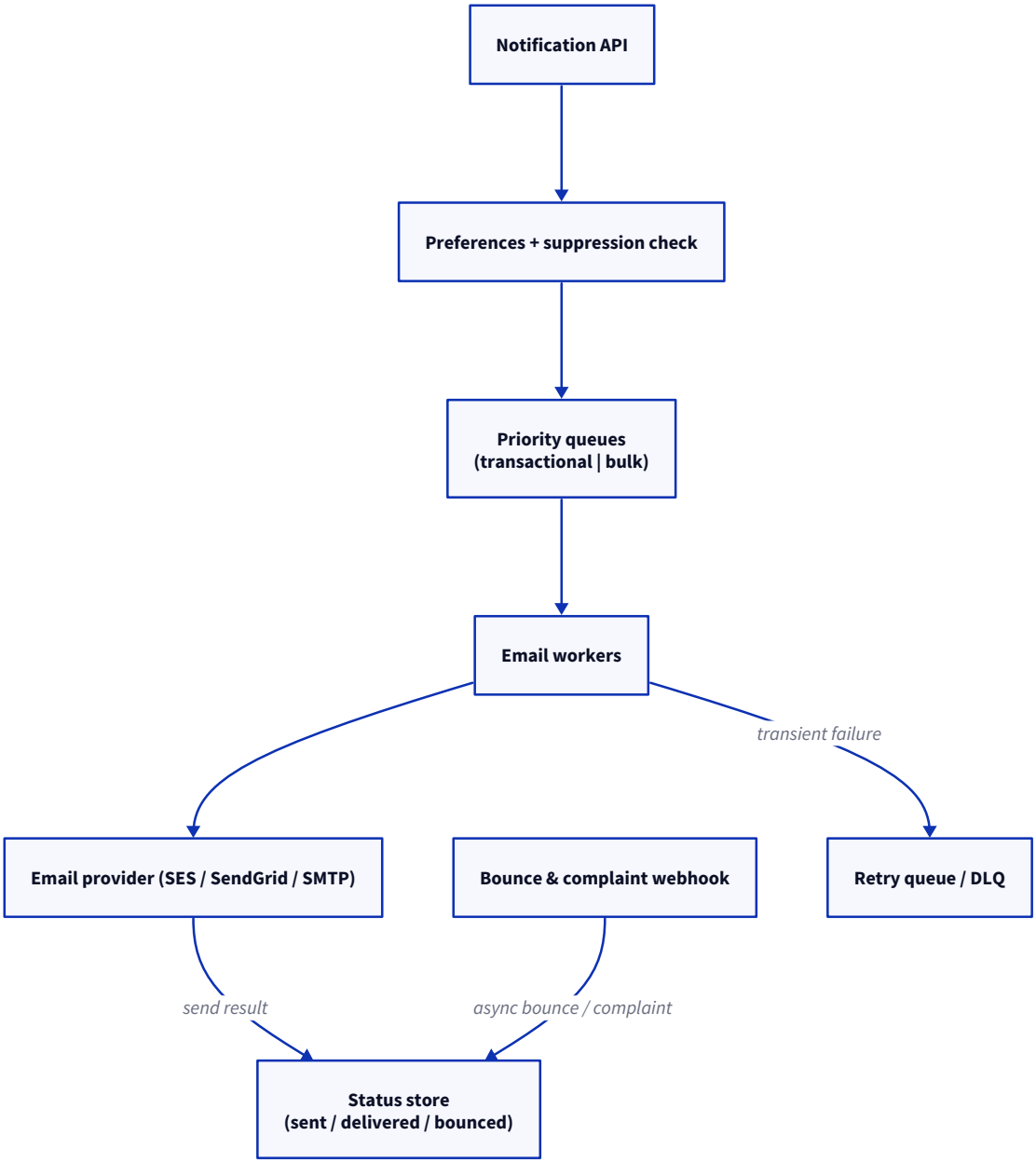
The shared pipeline (I set it up here). A notification API takes a send request, applies the user's preferences and a suppression check, and puts it on a channel queue; workers pick up the jobs and call the provider through a channel adapter; a status store tracks each message, and transient failures go to a retry queue or a DLQ. Email, SMS, and push all reuse this same shape and only swap the provider and the feedback path.

Delivery guarantees. Let me be honest that email is best-effort end to end. Your system can guarantee at-least-once handoff to the email provider, SES, SendGrid, or plain SMTP, but whether it actually lands in the inbox depends on spam filters and the recipient's server, and you learn about that asynchronously through the bounce and complaint webhooks. So "sent", meaning accepted by the provider, and "delivered", meaning confirmed, are two different states, and we track both.

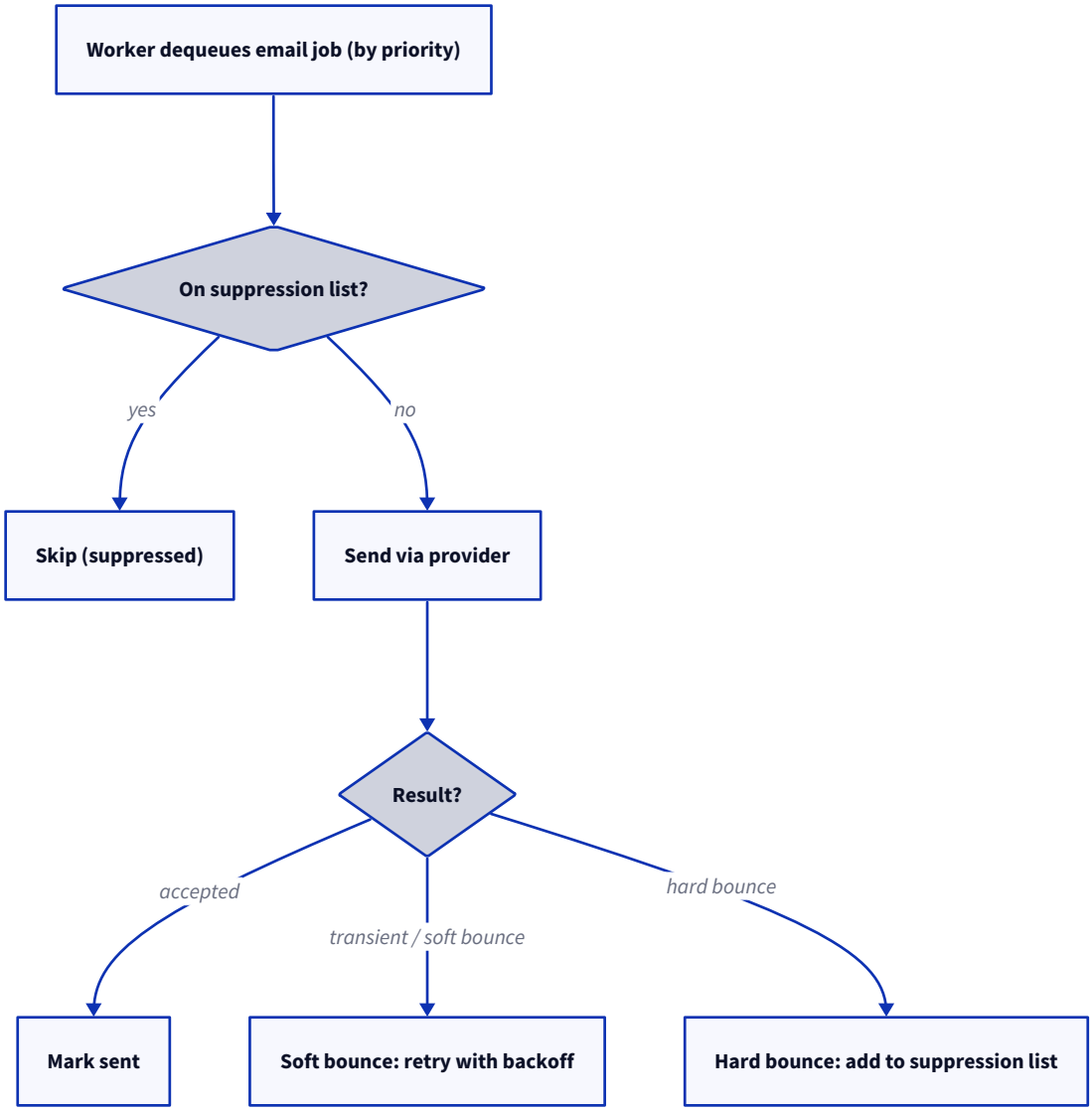
Retry strategies. We retry the transient failures, a provider 5xx or a temporary SMTP error, with exponential backoff. The email-specific nuance is the bounces. A soft bounce, like a full mailbox or greylisting, is worth retrying later, but a hard bounce, like an address that does not exist, must not be retried at all, and that address goes onto a suppression list so we never mail it again, since repeatedly mailing dead addresses quietly wrecks your sender reputation. And we use an idempotency key so that a retry never sends a duplicate.

Prioritisation. We keep the transactional email, the password resets and receipts that must arrive in seconds, in a separate queue from the bulk or marketing email, and we rate-limit the bulk lane. Otherwise a campaign of a million emails sits in front of somebody's password reset. This queue separation is the single most important design choice for a shared notification system.

High-level design.



Low-level design (send with bounce handling). A worker takes a job by priority, checks the suppression list and skips it if present, and sends through the provider. On acceptance it marks the message sent; a soft bounce goes back for a backoff retry; a hard bounce is added to the suppression list; and the asynchronous bounce or complaint webhook later reconciles the final delivered or bounced status.



Follow-ups they will push on.

- **Sent v/s delivered?** Sent means the provider accepted it; delivered is the later confirmation, or a bounce, through the webhook.
- **Why a suppression list?** Repeatedly mailing hard-bounced or complained addresses destroys sender reputation and deliverability.
- **How do you avoid duplicates on retry?** An idempotency key per message, deduped before sending.
- **Deliverability?** SPF, DKIM, and DMARC on your sending domain, plus honouring unsubscribes.
- **A huge campaign?** Rate-limit the bulk lane and shard the send, so transactional email is never blocked.

6. Design a notification system for SMS. Discuss delivery guarantees, retry strategies, and prioritization.

What this question is really testing. Whether you appreciate what makes SMS different from email: every message costs money, the carriers put strict rate limits, the delivery confirmation comes back as an

asynchronous and often unreliable carrier receipt (a DLR), and the top use case, OTP or 2FA, is latency-critical, since it must arrive within seconds. All of this changes how you think about retries and prioritisation. It reuses the pipeline from the email answer.

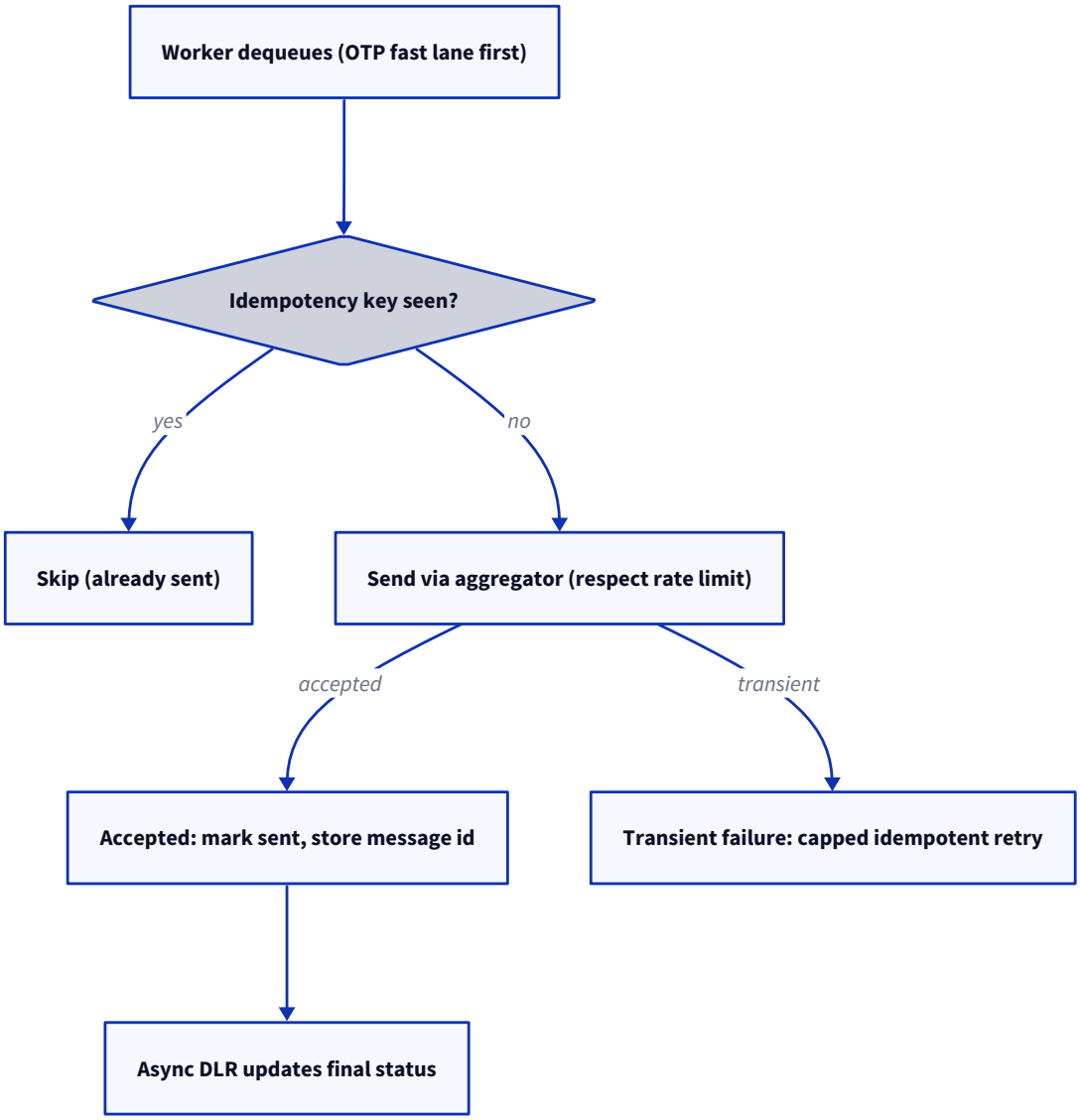
Delivery guarantees. Your system guarantees at-least-once handoff to the SMS aggregator, Twilio or a carrier gateway. The final delivery is confirmed by a DLR that the carrier sends back asynchronously, and since DLRs are sometimes delayed and sometimes never arrive, you treat delivery as best-effort confirmed: mark accepted when the aggregator takes it, and delivered only when the DLR says so.

Retry strategies. This is where SMS moves away sharply from email, because retries cost money and risk sending a duplicate text, which annoys the user and burns the budget. So you retry only on a clear transient failure, cap the attempts tightly, and lean hard on idempotency, a key per message so a retry cannot double-send. You must also respect the carrier rate limits by shaping the throughput on the workers, or the carrier will reject or throttle you.

Prioritisation. The OTP or 2FA traffic gets a dedicated fast lane and skips past everything else, because a 2FA code that arrives 30 seconds late is a failed login. Transactional alerts come next, and marketing is lowest and rate-limited. That fast lane for OTP is the defining prioritisation decision for SMS.

High-level design.

Low-level design (priority, idempotent send, DLR reconciliation). A worker pulls from the OTP fast lane first, checks the idempotency key and skips if it is already sent, and sends through the aggregator within the rate limit. On acceptance it stores the provider message id and marks the message sent; the asynchronous DLR later updates the final status; and a transient failure gets a capped, idempotent retry.



Follow-ups they will push on.

- **Why be so careful with retries?** Each SMS costs money, and a naive retry double-charges and double-texts, so idempotency is essential.
- **DLR never arrives?** Treat it as best-effort, time out to "unknown", and do not keep retrying on a missing receipt.
- **OTP latency?** A dedicated fast lane, the fewest hops, and a provider chosen for low-latency delivery in that region.
- **Carrier limits?** Shape the throughput per carrier and per number, and spread across numbers or providers if needed.
- **Compliance?** Honour opt-in, quiet hours, and per-country rules; sending without consent is a legal problem, not just a UX one.

7. Design a notification system for push. Discuss delivery guarantees, retry

strategies, and prioritization.

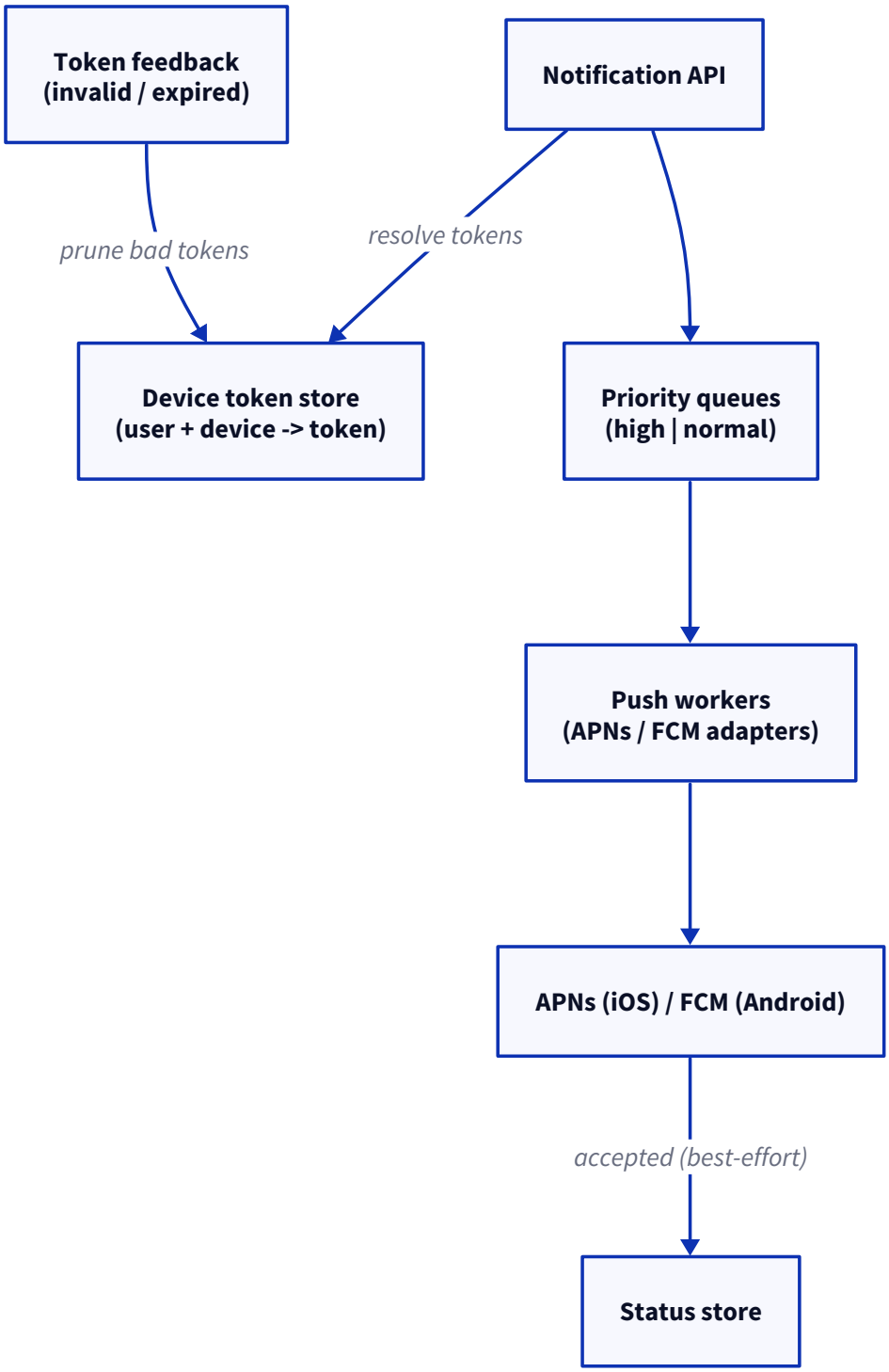
What this question is really testing. Whether you understand that push is best-effort by design, since Apple's APNs and Google's FCM do not guarantee delivery and give no read receipt, that the device tokens are the hard piece of state to manage, since they change, expire, and go invalid, and that the platform itself handles offline devices by queueing and collapsing. All of this makes retry and prioritisation look different from email and SMS. It reuses the pipeline from the email answer.

Delivery guarantees. Best-effort only, and I would say that plainly. You hand the message to APNs (for iOS) or FCM (for Android), which accept it, but they do not guarantee it reaches the device and they give no reliable confirmation that the user saw it. The device may be offline, in which case the platform queues it and may collapse older ones. So "accepted by APNs or FCM" is as far as your guarantee goes; "delivered" and "seen" are outside your hands.

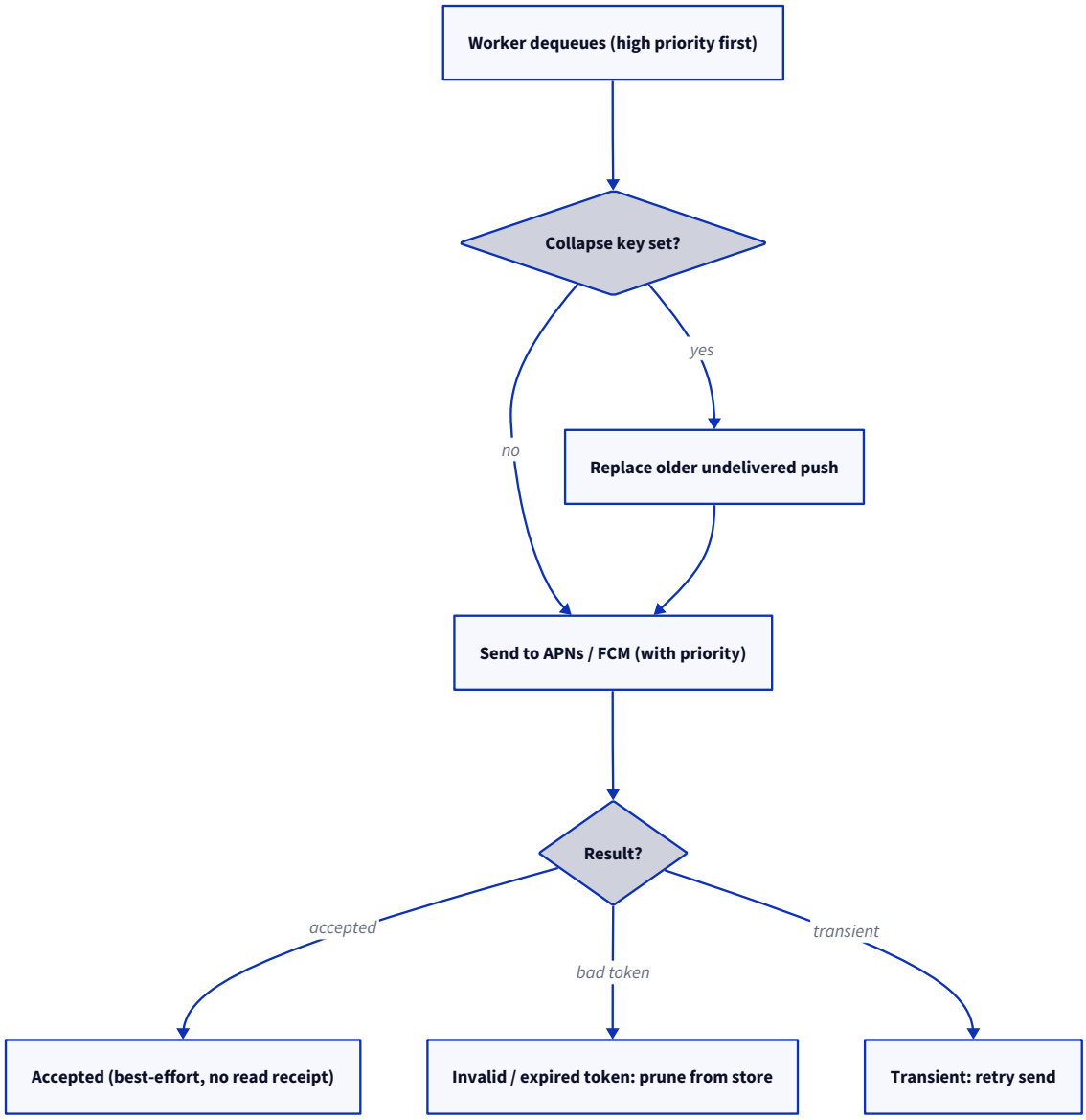
Retry strategies. Retrying has limited value here, because the platform already queues for offline devices, so you retry only on a transient error while talking to APNs or FCM, not to force delivery. The real work is token hygiene: APNs and FCM tell you when a token has gone invalid or expired, and you must prune it from your token store so you stop sending to dead devices. Collapsing matters too: for a "only the latest matters" notification, like a live score, you set a collapse key so a new push replaces an old undelivered one instead of stacking up.

Prioritisation. APNs and FCM expose priority levels: high priority wakes the device and delivers immediately, for the time-sensitive alerts, while normal priority is batched to save the battery. You also choose between an alert push and a silent or background one. So the prioritisation here is partly your own queues and partly a flag you set on the platform request.

High-level design.



Low-level design (send with token pruning and collapsing). A worker pulls the high-priority jobs first, applies a collapse key if one is set, replacing an older undelivered push, and sends to APNs or FCM with the chosen priority. On acceptance it records the best-effort status; an invalid or expired token is pruned from the store; and a transient error is retried.



Follow-ups they will push on.

- **How is push different from email and SMS?** It is best-effort with no read receipt, and the hard part is device-token management, not delivery confirmation.
- **Tokens change, how do you cope?** Store tokens per user and device, and prune them on the invalid or expired feedback from APNs and FCM.
- **Too many notifications?** Use a collapse key so only the latest of a kind is shown, and respect per-user rate limits and quiet hours.
- **High v/s normal priority?** High wakes the device for urgent alerts; normal is batched for the battery; do not mark everything high.
- **Silent pushes?** Background or silent pushes update the app state without alerting; the platform throttles them harder.